CareBike Project
Technical Study Guide

Real-time Booking • QR Code Identity • Repair & Loyalty Points

Prepared for: **Dang Quoc Khanh**

Scope: WebSocket/STOMP real-time, QR generate & scan, maintenance → loyalty pipeline

Stack: Spring Boot (Java) • React + TypeScript • Flutter (Dart)

A presentation & revision companion — every section maps to real code in the repository.

Theme: **Light**

Contents

1. Overview — the tech stack behind your part

Your three features span all three tiers of CareBike. They are the **connective tissue** between the apps: real-time booking, QR identity, and the repair→loyalty pipeline.

Tier	Technology	Your relevant files
Backend	Java + Spring Boot	backend-java/.../features/{appointment, maintenance, customer, vehicl
Web app (Branch/Admin)	React + TypeScript + Vite	web-app/src/pages/, services/
Mobile app (Customer)	Flutter (Dart)	mobile_app/lib/

The mental model: **Web (React)** and **Mobile (Flutter)** both talk to the same **Spring Boot backend** — over REST for actions, and over a WebSocket (STOMP) for live updates. The backend is the single source of truth.

> **How to read the file paths in this guide**
>
> Every Java file lives under the base folder `backend-java/src/main/java/com/carebike/backend/`, and its Java **package** mirrors that folder path (e.g. the folder `features/appointment/service` = package `com.carebike.backend.features.appointment.service`). To keep the tables short, that base is written as `backend-java/…/`. Mobile files are under `mobile_app/lib/` and web files under `web-app/src/`.

2. Feature 1 — Real-time booking (WebSocket / STOMP)

Where this lives in the repo

Role	File (path relative to repo root)
Backend — global config (package com.carebike.backend.config)	
WebSocket / STOMP broker config	`backend-java/…/config/WebSocketConfig.java`
Security (whitelists /ws handshake)	`backend-java/…/config/SecurityConfig.java`
Backend — appointment feature (package ….features.appointment)	
Controller (REST endpoints)	`backend-java/…/features/appointment/controller/AppointmentController.java`
Service (save + push over socket)	`backend-java/…/features/appointment/service/AppointmentService.java`
Entity (DB table)	`backend-java/…/features/appointment/entity/Appointment.java`
DTO (request body)	`backend-java/…/features/appointment/dto/AppointmentRequest.java`
Repository (DB queries)	`backend-java/…/features/appointment/repository/AppointmentRepository.java`
Mobile (Flutter)	
STOMP client service	`mobile_app/lib/services/web_socket_service.dart`
Customer booking screen	`mobile_app/lib/screens/customer_appointment_screen.dart`
Branch live board	`mobile_app/lib/screens/branch/branch_dashboard.dart`
Web (React + TypeScript)	
Appointment API calls	`web-app/src/services/appointmentService.ts`
Rescue live dashboard (STOMP)	`web-app/src/pages/RescueDashboard.tsx`
Admin live statistics (STOMP)	`web-app/src/pages/AdminDashboard.tsx`

Why WebSocket at all? (the core justification)

Normal REST/HTTP is **request–response**: the client asks, the server answers, the connection closes. The server has **no way to speak first**. For a booking board that must light up the moment a customer books, your REST options would be:

- **Polling** — the browser asks ‘any new bookings?’ every few seconds. Wasteful, laggy, hammers the server.
- **WebSocket** — open **one persistent two-way pipe**; the server pushes the instant something happens.

On top of WebSocket you use **STOMP** (Simple Text Oriented Messaging Protocol), which adds a **publish/subscribe channel system** — like radio stations. The server publishes to a channel; only clients subscribed to that channel receive it.

The full round-trip, step by step

Step 1 — Config turns on the message broker

WebSocketConfig.java

```
@EnableWebSocketMessageBroker
config.enableSimpleBroker("/topic");            // server -> client broadcast channels
config.setApplicationDestinationPrefixes("/app");
registry.addEndpoint("/ws").setAllowedOriginPatterns("*");  // handshake URL (web + mobile)
```

- **enableSimpleBroker("/topic")** = an in-memory broker built into Spring. It tracks who is subscribed to what and fans out messages.
- **setAllowedOriginPatterns("*")** = allow both the React web app and the Flutter mobile app (different origins) to connect.

Step 2 — Security lets the handshake through

SecurityConfig.java

```
.requestMatchers("/api/auth/**", "/error", "/ws/**").permitAll()
.sessionManagement(... SessionCreationPolicy.STATELESS)   // no server sessions - JWT only
.csrf(csrf -> csrf.disable())                             // required for mobile/React APIs
```

Step 3 — Customer books (REST POST), locked to customers

AppointmentController.java

```
@PreAuthorize("hasRole('CUSTOMER')")
public ResponseEntity<?> createAppointment(@RequestBody AppointmentRequest request) { ... }
```

Step 4 — Save + broadcast to the branch

AppointmentService.create()

```
Appointment savedAppointment = appointmentRepository.save(appointment); // status="PENDING"
String destination = "/topic/branches/" + branch.getId() + "/appointments";
messagingTemplate.convertAndSend(destination, savedAppointment);  // <- the real-time push
```

The whole appointment is serialized to JSON and pushed. The entity adds display fields via `@JsonProperty` (customerName, customerPhone, branchName) so the branch UI has everything without extra lookups.

Step 5 — The branch is subscribed and reacts instantly

web_socket_service.dart -> connectBranchAppointments()

```
destination: '/topic/branches/$branchId/appointments',
callback: (frame) {
  final newAppointment = jsonDecode(frame.body!);
  onAppointmentReceived(newAppointment);   // new card appears instantly, no refresh
}
```

Step 6 — Branch updates status → customer is notified

AppointmentService.updateStatus()

```
String customerDestination = "/topic/customers/" + customerId + "/appointments";
messagingTemplate.convertAndSend(customerDestination, updatedAppointment);
```

The customer's phone shows a SnackBar **and** pushes to a StreamController so the appointment list auto-reloads. Status is a simple state machine: **PENDING** → **CONFIRMED** → **COMPLETED** (or CANCELLED).

Two design details worth calling out

1. Per-user channels = privacy + routing

Channels are keyed by ID: `/topic/branches/{branchId}/appointments` and `/topic/customers/{customerId}/appointments`. A customer physically cannot receive another customer's updates because they never subscribe to that channel.

2. Separate STOMP clients + auto-reconnect = resilience

On mobile you keep two clients so opening one listener does not tear down the other. Every client sets `reconnectDelay: 5s` — if the network drops, it silently reconnects, then reloads the current truth from the database.

The same pattern is reused on web for the RescueDashboard and AdminDashboard (/topic/admin/stats makes the admin statistics count up live).

3. Feature 2 — QR code identity (generate + scan)

Where this lives in the repo

Role	File (path relative to repo root)
Mobile (Flutter)	
QR generate — customer (qr_flutter)	`mobile_app/lib/screens/tabs/profile_tab.dart`
QR scan — branch, camera (mobile_scanner)	`mobile_app/lib/screens/branch/branch_dashboard.dart`
Repair screen it opens into	`mobile_app/lib/screens/branch/create_maintenance_screen.dart`
Backend — vehicle feature (package ….features.vehicle)	
Plate-lookup endpoint (/lookup)	`backend-java/…/features/vehicle/controller/VehicleController.java`
Vehicle service	`backend-java/…/features/vehicle/service/VehicleService.java`
Vehicle entity	`backend-java/…/features/vehicle/entity/Vehicle.java`

The concept: a QR code is just a JSON envelope

A QR code is **not** an image of the customer — it is a **text container**. You put JSON in on one device and read the identical JSON out on another. No internet needed to decode; the data lives inside the code.

Generate — customer side

profile_tab.dart (package: qr_flutter)

```
final qrData = jsonEncode({
  'customerId': int.tryParse(userId) ?? 0,
  'fullName': name,
  'email': email,
});
QrImageView(data: qrData, version: QrVersions.auto, size: 180.0)
```

- **version: QrVersions.auto** = the library auto-picks a density big enough to fit the JSON.
- The dialog is branded ‘CareBike Identity Code’ and shows ID: CB-{userId} as a human-readable fallback.

Scan — branch side

branch_dashboard.dart (package: mobile_scanner)

```
final MobileScannerController _scannerController = MobileScannerController();
bool _isScanning = false;   // guard flag

void _onDetect(BarcodeCapture capture) async {
  final code = capture.barcodes.first.rawValue;
  if (code != null && !_isScanning) {
    setState(() => _isScanning = true);
    _scannerController.stop();                // freeze camera after first read
    final qrData = jsonDecode(code);          // <- exact reverse of jsonEncode
    final int customerId = qrData['customerId'];
    Navigator.push(... CreateMaintenanceScreen(customerId: customerId ...));
  }
}
```

> **Why the _isScanning guard matters (great detail to mention)**
>
> A camera fires the detect callback many times per second. Without the flag + `stop()`, you would open the maintenance screen 20 times. The flag makes it process exactly once, then restart on an invalid code — it shows you handled a real practical problem.

The payoff: the scanned customerId flows **directly into the repair form** — connecting Feature 2 to Feature 3. Staff never type an ID, so the charged/rewarded customer is guaranteed correct. (A plate-based fallback exists too: `GET /api/vehicles/lookup?licensePlate=...`)

4. Feature 3 — Repair → loyalty points → real-time notify

Where this lives in the repo

Role	File (path relative to repo root)
Backend — maintenance feature (package ….features.maintenance)	
Controller (POST /api/maintenance)	`backend-java/…/features/maintenance/controller/MaintenanceHistoryController.java`
Service (@Transactional pipeline)	`backend-java/…/features/maintenance/service/MaintenanceHistoryService.java`
Entity (DB table)	`backend-java/…/features/maintenance/entity/MaintenanceHistory.java`
DTO (request body)	`backend-java/…/features/maintenance/dto/MaintenanceHistoryRequest.java`
Backend — customer / loyalty feature (package ….features.customer)	
Loyalty service (points + tier math)	`backend-java/…/features/customer/service/LoyaltyService.java`
CustomerProfile entity (points/tier/spent)	`backend-java/…/features/customer/entity/CustomerProfile.java`
Profile controller (read profiles)	`backend-java/…/features/customer/controller/CustomerProfileController.java`
Profile repository	`backend-java/…/features/customer/repository/CustomerProfileRepository.java`
Mobile (Flutter)	
Repair form — branch	`mobile_app/lib/screens/branch/create_maintenance_screen.dart`
Loyalty card — customer	`mobile_app/lib/screens/tabs/profile_tab.dart`
Data models	`mobile_app/lib/models/loyalty.dart` , `mobile_app/lib/models/maintenance.dart`
Web (React + TypeScript)	
Customer + loyalty table (badges)	`web-app/src/pages/CustomerManagement.tsx`
Revenue charts (recharts)	`web-app/src/pages/Revenue.tsx`
Profile / maintenance API calls	`web-app/src/services/customerProfileService.ts` , `maintenanceService.ts`

The ‘one action, four effects’ pipeline

The whole feature hinges on **one method**, marked `@Transactional`:

MaintenanceHistoryService.create()

```
        @Transactional
        public MaintenanceHistory create(MaintenanceHistoryRequest request) {
            User customer = userRepository.findById(request.getCustomerId()).orElseThrow(...);
            MaintenanceHistory record = new MaintenanceHistory();
            record.setServiceDetails(...); record.setTotalCost(...); ...
            MaintenanceHistory saved = maintenanceRepository.save(record);        // (1) save history

            if (request.getTotalCost() != null)
                loyaltyService.addSpending(customer, request.getTotalCost());     // (2)+(3) points + tier

            String dest = "/topic/customers/" + customer.getId() + "/appointments";
            Map<String,Object> note = new HashMap<>();
            note.put("status", "COMPLETED");
            note.put("message", "Xe cua ban da duoc bao duong xong!");
            messagingTemplate.convertAndSend(dest, (Object) note);                // (4) notify live
            return saved;
        }
```

The input comes from the branch's form and matches the DTO exactly. Note `customerId` comes from the scanned QR:

create_maintenance_screen.dart

```
        await ApiClient.post('/maintenance', {
          'serviceDetails': ..., 'totalCost': ..., 'customerId': widget.customerId, // from the QR
        });
```

Why @Transactional is the star of this feature

> **Data integrity in one sentence**
>
> @Transactional wraps steps (1)-(3) in a single database transaction. If adding points throws an exception, the repair record insert is **rolled back too**. You never end up with a saved invoice but missing points, or vice-versa.

The loyalty math

LoyaltyService.addSpending()

```
        private static final BigDecimal POINTS_DIVISOR = new BigDecimal("100000"); // 1 pt / 100k VND

        profile.setTotalSpent(profile.getTotalSpent().add(invoiceAmount));          // cumulative spend
        int pointsEarned = invoiceAmount.divideToIntegralValue(POINTS_DIVISOR).intValue();
        profile.setAccumulatedPoints(profile.getAccumulatedPoints() + pointsEarned);
        profile.setMemberTier(calculateTier(profile.getTotalSpent()));             // re-rank tier
```

Tier ladder (by cumulative total spent):

Tier	Threshold (VND)
STANDARD	< 5,000,000
SILVER	≥ 5,000,000
GOLD	≥ 15,000,000
PLATINUM	≥ 30,000,000

- **getOrCreate()** lazily creates a CustomerProfile on the customer’s first repair (1-to-1 with User).
- **Guard:** a zero/negative invoice earns nothing but still returns the profile.
- **BigDecimal, not double** — money must be exact; double has floating-point rounding errors. Important detail.

Graphics / visualization of this data

- **Web (CustomerManagement.tsx):** loads users + profiles in parallel (Promise.all), builds a Map<userId → profile> for O(1) lookup, renders tier badges + points + a VIP counter. Revenue charts use recharts.
- **Mobile (profile_tab.dart):** a colored _LoyaltyCard, _StatBox for points/spend, and a _TierProgress bar toward the next tier.

5. How it all connects (one diagram)

Each feature feeds the next: **QR** → **repair form** → **loyalty** → **real-time notification**, all riding on the WebSocket backbone you built first.

```
          Customer books (mobile)  --REST-->  Backend  --WebSocket-->  Branch sees it live   [F1]
                                                  |
          Customer shows QR  -->  Branch scans (camera)  -->  pre-fills maintenance form   [F2]
                                                  |
          Branch saves repair  -->  @Transactional:  save history + add points + recalc tier
                                                   + WebSocket "Done!" back to customer     [F3]
                                                  |
          Result  -->  Loyalty card / tier badges update on web + mobile        [graphics]
```

> **The 30-second summary you can memorize**
>
> ‘A customer books on their phone; the branch sees it instantly over a WebSocket. When they arrive, they show a QR code that carries their ID; the branch scans it and the repair form is pre-filled. Saving the repair atomically stores the history, awards loyalty points, upgrades the membership tier, and pushes a live “done” notification back to the customer. All the loyalty data then renders as badges and charts on both apps.’

6. Q&A — likely questions & how to answer them

On WebSocket / real-time

Q: Why WebSocket instead of just refreshing or polling?

A: Polling wastes bandwidth and adds delay — you ask the server every few seconds even when nothing changed. WebSocket keeps one open connection and the server pushes only when there is a real event, so it is instant and efficient.

Q: What is STOMP and why not raw WebSocket?

A: Raw WebSocket is just a dumb pipe of bytes. STOMP adds a messaging layer with named channels (topics) and subscribe/publish semantics, so I can route messages — e.g. only branch #3 gets branch #3's bookings — without writing that logic by hand.

Q: How do you ensure one customer cannot see another's data over the socket?

A: Each user subscribes only to a channel keyed by their own ID, like /topic/customers/5/appointments. The server publishes to that specific channel, so messages are scoped per user.

Q: What happens if the internet drops mid-session?

A: Every STOMP client sets reconnectDelay of 5 seconds, so it automatically re-connects and re-subscribes. Because state lives in the database, the UI reloads the current truth once back.

Q: Is the WebSocket secure / authenticated?

A: The data-changing actions (booking, status change) go through JWT-protected REST endpoints with @PreAuthorize role checks. The /ws handshake is permitted so the socket can open; the sensitive actions are all guarded on the REST side. A hardening step would be authenticating the STOMP CONNECT frame with the JWT too.

Q: Where is the message broker — did you use RabbitMQ?

A: No — I used Spring's built-in simple in-memory broker (enableSimpleBroker), which is perfect for this scale. I could switch to RabbitMQ/ActiveMQ for horizontal scaling without changing the application code.

On QR code

Q: What is actually inside the QR code?

A: JSON text: {"customerId": 5, "fullName": "...", "email": "..."}. I encode it with jsonEncode on the phone and jsonDecode it after scanning. The QR is just a carrier for that string.

Q: Isn't putting customer data in a QR a privacy risk?

A: It only holds an ID, name, and email — no password, no payment info. It is shown deliberately by the customer to staff, like a membership card, and the ID is only useful to someone already authenticated in the branch app.

Q: Why does the camera not open the screen multiple times?

A: A scanner fires the detect callback many times per second. I use an _isScanning boolean flag and stop the controller after the first successful read, so it processes exactly once, then restarts on an invalid code.

Q: What if someone scans a random/invalid QR?

A: jsonDecode fails or customerId is missing — I catch that, show an 'Invalid QR code' message, and restart the scanner.

On repair + loyalty

Q: What does @Transactional do here and why does it matter?

A: It wraps saving the repair record and updating loyalty points into one atomic database transaction. If any step fails, everything rolls back — so I can never save an invoice without its points, or add points for a repair that did not save. It guarantees consistency.

Q: Why BigDecimal instead of double for money?

A: double uses binary floating point and produces rounding errors (0.1 + 0.2 is not 0.3). For currency that is unacceptable, so I use BigDecimal, which is exact.

Q: How are points and tier calculated?

A: 1 point per 100,000 VND spent. Tier is based on cumulative total spend: Silver at 5M, Gold at 15M, Platinum at 30M. After each repair I add the amount to totalSpent, add points, then recompute the tier.

Q: When is the customer's loyalty profile created?

A: Lazily — getOrCreate() makes it on the customer's very first repair. New customers do not carry an empty profile until they actually transact.

Q: How does the customer find out the repair is done?

A: The same WebSocket. After saving, the backend pushes a {status: COMPLETED, message: ...} event to that customer's channel and their phone shows it live. So Feature 3 reuses Feature 1's infrastructure.

General / architecture

Q: Why is the backend split into features/appointment, features/maintenance, etc.?

A: It is feature-based (vertical slice) packaging — each feature keeps its controller, service, entity, repository, and DTO together. Easier to navigate and scales better than grouping by technical layer.

Q: What is the difference between an Entity and a DTO?

A: The Entity (e.g. Appointment) maps to a database table. The DTO (e.g. AppointmentRequest) is the shape of the incoming JSON request — it carries only what the client should send, decoupling the API from the database schema.

Q: How do the three apps talk to each other?

A: Everything goes through the Spring Boot backend. Web (React) and mobile (Flutter) both call the same REST API for actions and subscribe to the same STOMP topics for live updates. The backend is the single source of truth.

Q: What was the hardest part?

A: Making the real-time flow bidirectional and reliable — routing messages to the right per-user channel, keeping two STOMP clients from killing each other on mobile, and chaining the repair → points → notify

steps atomically so nothing gets half-done.

7. Glossary of key terms

Term	Plain-English meaning
REST / HTTP	Request-response API. Client asks, server answers, connection closes.
WebSocket	A persistent, two-way connection so the server can push data without being asked.
STOMP	A messaging protocol over WebSocket that adds named channels (topics) + subscribe/publish.
Topic / channel	A named stream like /topic/customers/5/appointments that clients subscribe to.
Broker	The component that tracks subscriptions and fans messages out. Here: Spring in-memory broker.
JWT	A signed token proving who the user is; sent with each request instead of a server session.
@PreAuthorize	Spring annotation that restricts an endpoint to a role, e.g. hasRole('CUSTOMER').
Entity	A Java class mapped to a database table (JPA).
DTO	Data Transfer Object - the shape of a request/response, separate from the DB entity.
@Transactional	Runs a group of DB operations as all-or-nothing; failure rolls everything back.
BigDecimal	Exact decimal type used for money to avoid floating-point rounding errors.
qr_flutter	Flutter package that renders a QR image from a string.
mobile_scanner	Flutter package that reads QR/barcodes from the live camera.
recharts	React charting library used for the revenue/statistics graphics.
Lazy creation	Create a record only when first needed (e.g. loyalty profile on first repair).

End of guide. Every code snippet above is a simplified excerpt of real files in the CareBike repository — open the referenced file to see the full context during your presentation.